

VEMO — Critical Review & Scorecard

An adversarial audit of the rebuilt governance framework, the shipped implementation (not just the design), and a v1.1 hardening round.

Subject **VEMO v1.0 → v1.1**

Lens **adversarial auditor**

North star **time-to-correct-commit**

Date **2026-06-16**

8.6

DESIGN INTENT
(ITERATION LOG)

7.3

SHIPPED V1.0
(AUDITED, STRICTER)

8.3

V1.1
(AFTER HARDENING)

■ HEADLINE

The design *intent* scored 8.6 in the iteration log. This review audits the **shipped code** with a skeptic's eye and is deliberately stricter: VEMO v1.0 had the right architecture but three enforcement mechanisms were **holed** — a parser bug left the R2 judge gate **silently dead**, the risk tier was **self-declared** (an agent could downgrade R2 → R0 to skip gates), and degradation was **fail-open**. A framework whose "hard gates" don't actually fire is back to Wildpanda's problem in a nicer costume. **v1.1 repairs all three** (verified, below) and lifts the shipped score to 8.3. The remaining gap to the design intent is honest: Windows hook portability and a real eval runner.

🔄 UPDATES SINCE THIS REVIEW (V1.2 → V1.4)

This review audits **v1.0 → v1.1**; the framework has since continued. **v1.2**: a full-auto/unattended mode (records every decision; safety + judge unchanged; explicit-command-only, TTL-bounded). **v1.3** (from Fable 5 / Mythos analysis — see FABLE5_MYTHOS_OPTIMIZATION.md): **run-budget stop rules** (hard-stop a runaway when unattended, advisory when attended), a **frontier** capability tier + frontier

model routing, and a judge **evidence-completeness** check (guards the documented 20× undercount).
v1.4: a unified `vemo` CLI, per-stack **presets**, and Diátaxis docs — the usability/understandability/generalization round. Full history: CHANGELOG.md.

1 What VEMO is (English overview)

VEMO (**Velocity-first, Enforced, Model-aware Orchestration**) is a drop-in governance framework for AI coding agents, rebuilt from Wildpanda + 2026 best practices. Its thesis: keep Wildpanda's philosophy, but pay for governance with **mechanism** instead of **developer speed**.

Five shifts vs. Wildpanda

#	Wildpanda	VEMO
1	Gates are prose the model may ignore	Mechanically enforced (hooks + CI backstop) — "can't hallucinate" ^[1]
2	Front-load ~20 specs each session	JIT-load only what a task needs (manifest) ^[5]
3	One ceremony level for everything	Risk-tiered : R0 fast / R1 standard / R2 critical
4	Ceremony is a constant	Scales with model capability (<code>capability.tier</code>) ^[2]
5	Agent self-reports a gate pass	Independent judge subagent verifies high-risk gates

Architecture (four layers + one new enforcement layer)

Entry

AGENTS.md — a ~50-line router (vs Wildpanda's 215). Loads config + the always-on safety spec, then JIT-loads the rest.

Config

vemo.config.yaml — the single instance file: `capability.tier`, `model_routing`, `risk_tiers`, `enforcement`.

Specs

specs/ — slim, JIT-loaded (task / safety / verify / concurrency / coding) + `_manifest.yaml` (progressive disclosure).

Enforce

enforcement/ — *the layer Wildpanda lacked*: Claude Code hooks (client-side), git pre-commit + CI (server-side authority), and a stdlib-Python validator that parses machine-readable task front-matter.

Agents

agents/governance-judge.md — an independent verifier (Opus) prompted to refute "done"; **eval/** — a conformance harness.

The velocity engine: risk tiers

R0 trivial · plan-inline → implement → done

R1 standard · plan → implement → verify

R2 critical · plan → review → implement → verify(+judge) → procedure

A typo and a schema migration do not deserve the same process. Default to the lowest tier that fits — that is where the developer-acceleration comes from.

2 Critical review (adversarial auditor lens)

I assume the worst: a lazy or adversarial agent, a missing tool, a tired developer at 6pm. The question is not "is the design nice?" but "what actually fires, and what can be bypassed?"

⚙️ DEMONSTRATED DEFECT — THE R2 JUDGE GATE WAS SILENTLY DEAD IN V1.0

VEMO v1.0 used a hand-rolled stdlib YAML reader. Run against a real task file it **collapsed nested mappings to []**, so `judge.verdict` was unreadable — the gate that is supposed to make the judge mandatory could never see a verdict:

```
# v1.0 parser on tasks/T-...-demo.md
parsed keys: [... 'acceptance', 'status', 'build_exit', ... 'judge', 'required', 'verdict']
acceptance value -> []      # should be a mapping
judge value      -> []      # judge.verdict NEVER readable → R2 judge gate is dead
```

This is the cardinal sin a VEMO review must catch: a "hard gate" that doesn't fire is exactly the failure VEMO was built to fix. Found, then fixed in v1.1 (\$3).

Findings

ID	Severity	Finding	v1.1
F1	High	Parser correctness: nested front-matter collapsed → <code>judge.verdict</code> unreadable → R2 judge gate dead. (demonstrated above)	fixed
F2	High	Self-classification loophole: risk tier was agent-declared. A lazy/adversarial agent marks an R2 change "R0" and skips review + judge. The trust problem VEMO criticized, reintroduced.	fixed
F3	High	Judge not mechanically required: nothing forced the judge subagent to actually run on R2 — the agent could simply not invoke it.	fixed
F4	Med	Fail-open degradation: <code>degrade_gracefully:true</code> meant a missing python3 silently disabled the scope guard (logs, allows). A missing guard treated as a safe guard.	fixed
F5	Med	Windows portability: hooks are bash + python3; on Windows they need git-bash/WSL — ironically weaker than Wildpanda's PowerShell in the <i>enforcement</i> path.	partial
F6	Low	eval/ is aspirational: ships scenarios + metric definitions but no agent-running harness — measurability is promised, not yet executable.	roadmap
F7	Low	No config validation: a typo in <code>vemo.config.yaml</code> (e.g. <code>tier: hihg</code>) silently falls back.	fixed
F8	Inherent	Irreducible trust core: the session-start bootstrap ("load config + manifest") is still advisory prose. Enforcement <i>shrinks</i> the trust surface; it cannot remove the first step where the agent must choose to read.	accepted

© WHAT HOLDS UP WELL

Philosophy & problem framing, the risk-tiered velocity model, JIT context, `capability.tier` +routing, and the kept-from-Wildpanda concurrency/takeover model are all genuinely strong and survive the audit unchanged. The weaknesses were concentrated in the **correctness and bypass-resistance of the enforcement mechanisms** — which is precisely where a governance framework must be unimpeachable.

3 Scorecard (shipped v1.0 → v1.1)

Same rubric used for the original Wildpanda analysis (so it stays comparable: Wildpanda = 6.1). Scored against the shipped implementation, not the design deck.

Dimension	v1.0 → v1.1		Note
Problem framing / philosophy	9	→ 9	Unchanged — strong.
Enforcement: correctness	5	→ 9	F1 parser bug → fixed; gates now actually read state.
Enforcement: bypass-resistance	5	→ 9	F2/F3/F4 → diff-derived tier, required judge, fail-closed.
Developer velocity	9	→ 9	R0 fast path intact; enforcement adds no friction to low-risk work.
Context engineering	8	→ 8	JIT manifest + thin AGENTS.md.
Model-capability adaptation	8	→ 8	tier + routing; unchanged in v1.1.
Reusability & decoupling	8	→ 8	Single config; template/instance split.
Concurrency safety	9	→ 9	Kept from Wildpanda + CI ownership check.
Observability & eval	6	→ 7	Telemetry + doctor ; eval runner still pending (F6).
Cross-platform / portability	5	→ 6	Core logic pure-Python; bash hooks still need git-bash on Windows (F5).
Maintainability & DX	7	→ 8	CHANGELOG + doctor + spec line-caps.
Standards alignment	8	→ 9	AGENTS.md + hooks + subagents, all in-layer ^{[1][4]} .
Composite	7.3 → 8.3		vs Wildpanda 6.1

Each fix maps to a finding and is backed by a run, not a claim. v1.1 makes the enforcement layer **correct and non-bypassable**.

Fix	Finding	What changed
Correct nested parser	F1	New indentation-based YAML-subset parser in <code>task_state.py</code> ; <code>judge.verdict</code> is readable again. Also caught a follow-on bug: inline lists with trailing <code>#comments</code> were parsed as character strings (fixed via <code>rfind("]")</code>).
Diff-derived risk tier	F2	<code>tier-required</code> computes the required tier from the staged paths; the CI backstop blocks a self-downgrade (declared < required).
Mandatory R2 judge	F3	CI blocks an R2 push unless <code>judge.verdict == pass</code> . The judge can no longer be skipped by not calling it.
Fail-closed guards	F4	<code>degrade_gracefully</code> now defaults false ; scope/command/secret guards block when they cannot verify.
<code>doctor</code> + nested <code>get</code>	F7	Config + task front-matter validation; dotted field reads enable the gates above.

✓ VERIFICATION — THE FIXES ACTUALLY WORK (LIVE RUNS)

```
# F1 – nested parsing restored
acceptance -> {'status': 'not_run', 'build_exit': None, ...}
acceptance.status -> not_run          judge.verdict -> None      # readable now

# F2 – required tier computed from PATHS (not the agent's word)
README.md      -> R0      src/app/x.ts      -> R1
src/core/k.cpp -> R2      mixed(R0+R2)      -> R2  # takes the highest
# → CI blocks if a task declaring R0 stages src/core/** (downgrade refused)

# F4 – scope guard still blocks out-of-scope edits
edit src/auth/login.ts (out of task scope) -> hook exit 2 (BLOCKED)
edit src/fusion/core.ts (in scope)         -> hook exit 0 (allowed)

# F7 – config/task sanity
VEM0 doctor: OK – no issues
```

Net effect: the three High-severity findings (F1–F3) and one Medium (F4) are closed and demonstrated; the enforcement layer now does what the design promised — it fires, and it cannot be talked out of firing.

5 Remaining limitations & roadmap (honest)

Item	Sev	Plan
Windows hook runner (F5)	Med	Ship a pure-Python hook dispatcher (or Node) so hooks run natively on Windows without git-bash. Core logic is already pure-Python.
Executable eval harness (F6)	Low	A runner that drives an agent through each scenario and asserts the gate outcome from telemetry + front-matter; report per-model-tier metrics so relaxing a tier on a stronger model is a data decision.
Tamper-evidence	Low	CI verifies the hook/CI config + validator hashes are intact, so "disable the guard" is itself detectable.
Irreducible bootstrap (F8)	Inherent	Cannot be hooked away (the agent must choose to read step 1). Mitigation: keep it one line; make everything after it mechanical. State the limit honestly rather than pretend.

🔒 CLOSING

The discipline this review models is the point: a governance framework earns trust only by being **audited adversarially against its own promises**. VEMO's promise is "gates that fire." v1.0 made that promise in prose-free mechanism but didn't fully keep it; v1.1 does, and says clearly where it still falls short. That loop — build, attack, measure, fix, disclose — is itself the most important best practice, and it is the one VEMO bakes into `eval/` so it keeps happening as models get stronger.

Sources / best practices referenced: [\[1\] Claude Code hooks/subagents/skills \(2026\)](#) · [\[2\] Spec-driven dev / Kiro Auto router](#) · [\[3\] 12-Factor Agents](#) · [\[4\] AGENTS.md standard](#) · [\[5\] Anthropic — context engineering](#)

VEMO · Critical Review & Scorecard (EN) · 2026-06-16 · auditor lens: Claude (Opus 4.8)

Framework: [VEMO/README.md](#) · Design log: [ITERATION_LOG.html](#) · Changelog: [CHANGELOG.md](#)